

# Automaten, Sprachen und Compiler

Prof. Dr. Uwe Meyer

# Agenda


| | | |
|---|---|---|
| **1.** | Einleitung | 2 |
| **2.** | Grundlagen | 19 |
| **3.** | Programmiersprachen | 31 |
| **4.** | Automaten | 45 |
| **5.** | Grammatiken | 109 |
| **6.** | Syntaxanalyse | 150 |

# Klausur

- In der Klausur sollen Sie nachweisen, dass Sie die notwendigen **Kompetenzen besitzen.**
- Auswendiglernen ist **keine** solche Kompetenz!
- Sie dürfen bei der Klausur einen handgeschriebenen selbstgeschriebenen, beidseitigen DIN A4-Zettel benutzen.

- D.h., die Klausuraufgaben zielen nicht auf die Abfrage von Wissen, sondern auf das Verständnis, die Anwendung und den Transfer.

- Sie müssen den Stoff **verstehen**!



# Vorlesung und Übung

## Vorlesung

- Die Folien zur Vorlesung werden i.A. vorher auf Moodle hochgeladen.
- **Schreiben Sie mit** (per Hand!) – das hat den höchsten Lerneffekt.
- Rechnungen und Beispiele werden oft auf die Tafel geschrieben – schreiben Sie diese auf jeden Fall auch mit!
- Nach der Vorlesung sollten Sie die Vorlesung **nachbereiten**, d.h. den Stoff durchgehen, ggf. in der Literatur nachlesen und die Beispiele nochmal **selbst ausprobieren**.
- Die Folien sind tw. auch das Skript – wir werden nicht jede Folie im Detail besprechen.

## Praktikum

- In den Praktika werden Sie von Mitarbeitern und Tutor:innen in kleineren Gruppen betreut.
- Der Fokus liegt auf „Üben“, nicht „Vorlesung nochmal wiederholen“.
- Die Aufgaben werden besprochen – Sie stellen Ihre Lösungen vor.
- Notieren Sie sich in der Nachbereitung die Punkte aus der Vorlesung, die Ihnen noch nicht klar sind, und fragen Sie die Tutorinnen und Tutoren!

# Organisatorisches



- Vorlesung: Montags, 4. Block
- Es werden 3 Praktikumstermine Dienstags 2.-4. Block angeboten.
- Tragen Sie sich in 3-er Gruppen ein(Gruppenwahl „Praktikumsgruppen“ – ab 16.10., 9:00 Uhr bis inkl. 22.10.).
- Wenn Sie nicht in eine Gruppe eingetragen sind, können Sie keine Hausaufgaben abgeben und **nicht** die Klausurzulassung erreichen!

# Prüfung

- Klausur im zweiten Prüfungszeitraum, d.h. wahrscheinlich Ende März.

- Voraussetzung für die Zulassung zur Klausur ist:
  - Korrekte Lösung der Praktikumsaufgabe.
    Die Praktikumsaufgabe besteht aus drei Teilen, deren Lösung Sie durch die Mitarbeiter oder Tutoren abnehmen lassen müssen. Pro Teilaufgabe haben Sie maximal 2 Versuche und jedes Gruppenmitglied muss in der Lage sein, den Transfer von der Theorie zu Ihrem Code zu erklären. Details zur Praktikumsaufgabe finden Sie in Moodle.
  - Zusätzlich wird eine Bonusaufgabe gestellt, für deren Lösung Sie bis zu 6 Punkten für die Endnote erreichen können.
- Wir werden Ihnen Teile des Codes vorgeben. Dieser Code ist umfangreich und komplex, sodass der Besuch des Praktikums dringend angeraten ist, um die Aufgaben bearbeiten zu können!
- Die Note wird allein durch das Klausurergebnis bestimmt (+ evtl. Bonuspunkte).

# Warum Informatik studieren, statt ChatGPT zu nutzen?

**Grundlagen**: KI Tools basieren auf Algorithmen, Datenstrukturen, theoretischen Grundlagen und System Design.
Ohne dieses Wissen können Sie nicht
- **Korrektheit** der Antworten der KI **beurteilen**
- **Fehler** oder Inkonsistenzen **debuggen**
- **Lösungen** an neue Probleme **anpassen**.

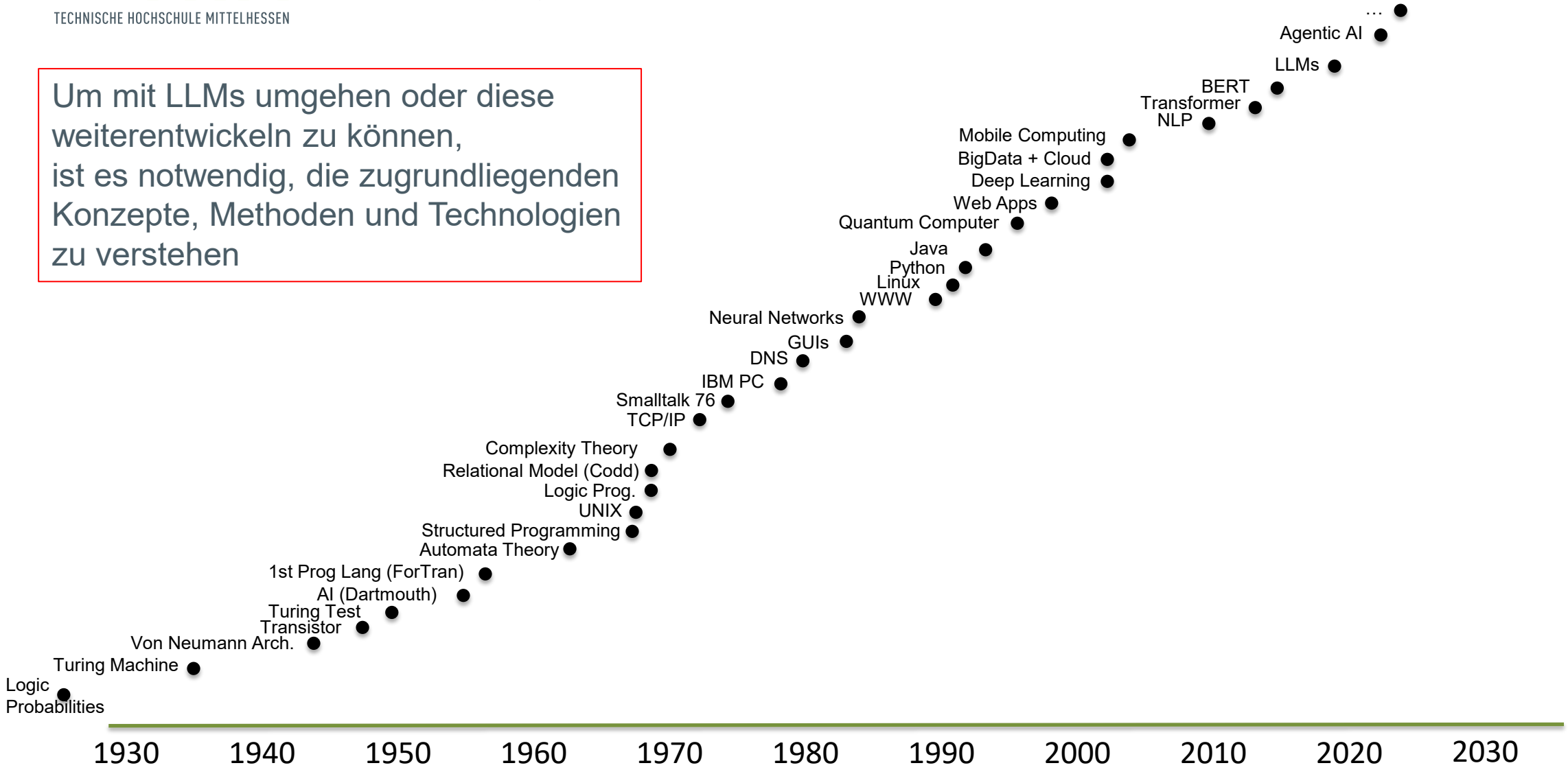
**Grenzen der KI:** Im Informatikstudium lernen Sie die Grenzen der KI zu erkennen:
- KI kann **halluzinieren** und falsche Ergebnisse produzieren
- In Spezialgebieten sind **Daten** für das Training der KI oft **nicht verfügbar**.
- KI **scheitert** oft an Aufgaben, die formale Korrektheit erfordern (z.B. sicherheitskritische Systeme, Kryptoprotokolle)

**Kritisches Denken und Problemlösungen: I**m Informatikstudium lernen Sie
- **Abstraktes Denken** (wie modelliert man Problemstellungen?)
- **Algorithmisches Denken** (wie entwirft man korrekte und effiziente Lösungen?)
- **Grenzen der Berechenbarkeit** (was kann man berechnen und was kann effizient berechnet werden)?

**Ethik, Sicherheit und Privacy:** Sie lernen
- **Verantwortungsvollen** Umgang mit Technologie
- Entwicklung von Systemen, die **sicher ggü. zukünftigen Bedrohungen** sind
- **Interpretation** von ethischen Leitlinien und Gesetzen

**Das alles kann KI nicht!**

Um mit LLMs umgehen oder diese weiterentwickeln zu können,
ist es notwendig, die zugrundliegenden Konzepte, Methoden und Technologien zu verstehen



# MIT Studie zu Auswirkungen der Nutzung von KI auf das Gehirn (https://arxiv.org/pdf/2506.08872)

## Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task△

**Nataliya Kosmyna** [1]
*MIT Media Lab*
*Cambridge, MA*

**Eugene Hauptmann**
*MIT*
*Cambridge, MA*

**Ye Tong Yuan**
*Wellesley College*
*Wellesley, MA*

**Jessica Situ**
*MIT*
*Cambridge, MA*

**Xian-Hao Liao**
*Mass. College of Art and Design (MassArt)*
*Boston, MA*

**Ashly Vivian Beresnitzky**
*MIT*
*Cambridge, MA*

**Iris Braunstein**
*MIT*
*Cambridge, MA*

**Pattie Maes**
*MIT Media Lab*
*Cambridge, MA*

*„… over the course of 4 months, the LLM group's participants performed worse than their counterparts in the Brain-only group at all levels."*



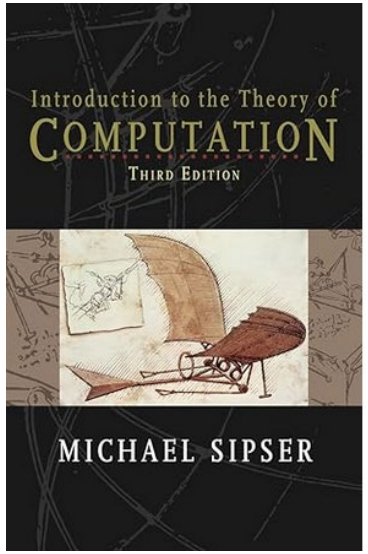

# Wichtig!

- Während der Klausur dürfen Sie ChatGPT o.ä. nicht benutzen!
- Dementsprechend müssen **Sie** den Stoff verstanden haben – ob ChatGPT den Stoff versteht, ist irrelevant.
- Daher gelten folgende Regeln:
  - Die Benutzung von KI Tools (ChatGPT, CoPilot, …) zur Erstellung von Code während der Praktika ist **verboten**. Einmalige Benutzung wird mit einer gelben Karte geahndet, bei zwei gelben Karten erhalten Sie keine Klausurzulassung.
  - Die Benutzung von KI-Tools für die Bearbeitung der Praktikumsaufgabe ist verboten. Sollten Sie den Code in Ihren Lösungen nicht erklären können, erhalten Sie keine Klausurzulassung.

- Der Sinn einer Ausbildung ist es, Fähigkeiten zu erlernen. Lernen kann man nicht durch Abschreiben!
- Zu einigen Praktikaterminen werden Übungsaufgaben auf Papier verteilt. Diese sollen Sie in einer größeren Gruppe ohne KI lösen.

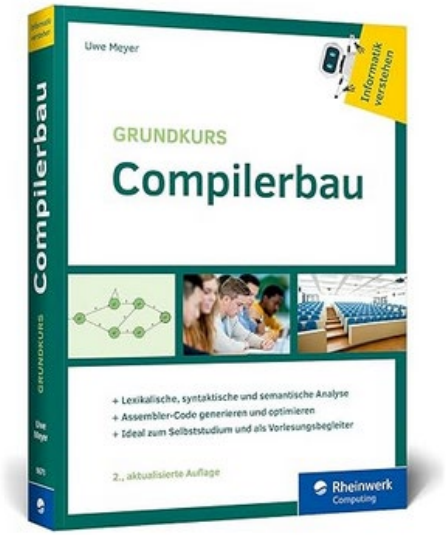
# Literatur

- Sipser, Michael. Introduction to the Theory of Computation.




https://hds.hebis.de/thm/Record/HEB519982649

- Meyer, Uwe. Grundkurs Compilerbau




https://hds.hebis.de/thm/Record/HEB479924139

# Ziele und Inhalte (aus Modulhandbuch)

***Qualifikations- und Lernziele***

Die Studierenden kennen die Bedeutungen und Zusammenhänge von regulären Ausdrücken,

endlichen Automaten, Kellerautomaten, Turingmaschinen, Berechenbarkeit, formalen Sprachen,

Grammatiken und deren Einsatz in den Phasen der lexikalischen Analyse und der Syntaxanalyse in

einem Compiler.

***Lerninhalt 2***

- Reguläre Ausdrücke und endliche Automaten
- Reguläre und kontextfreie Sprachen und Grammatiken, Chomsky-Hierarchie, Kellerautomat
- Kontextsensitive, Turing Maschine, Turing-erkennbare und -entscheidbare Sprachen
- Transformationen zwischen diesen Konzepten
- Pumping-Lemmata
- Erstellung eines Lexers mit einem Generator
- Syntaxanalyse top-down und bottom-up
- Erstellung eines Parsers mit einem Generator
- Abschlusseigenschaften der Sprachen

# Teilnahmevoraussetzungen

| PO 2010/2016 | PO 2022 |
| --- | --- |
| Inhaltliche Voraussetzungen:<br>- Algorithmen und Datenstrukturen<br>- Programmieren interaktiver Systeme<br><br>Wer eine oder beide nicht erfüllt, kommt bitte in meine Sprechstunde (Dienstag, 13 Uhr). | Notwendige Voraussetzung:<br>Mindestens 42 Credit Points aus der Grundlagenphase (1. + 2. Semester).<br><br>Wer dies nicht erfüllt, kann nicht an dem Modul teilnehmen. |

# Die drei Säulen der theoretischen Informatik

## Automaten und Sprachen

- Eigenschaften mathematischer Modelle von „Sprache“
- Wie kann man erkennen, ob ein Programm syntaktisch korrekt ist?
- Entwickelt, um Compiler bauen zu können.

## Berechenbarkeit (*computability*)

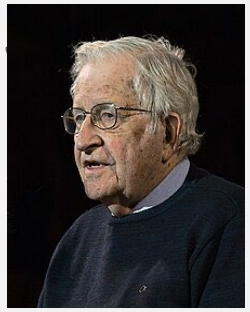
- Ist eine mathematische Aussage wahr oder falsch?
- Theoretische Modelle von Computern
- Welche Probleme sind durch Computer lösbar, welche nicht?

## Komplexität

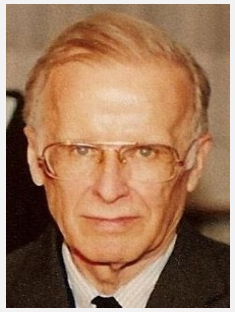
- Wie schwierig ist es, Probleme mit Computern zu lösen?
- Wie ist die Laufzeit von Algorithmen?

# Die drei Säulen der theoretischen Informatik

## Automaten und Sprachen

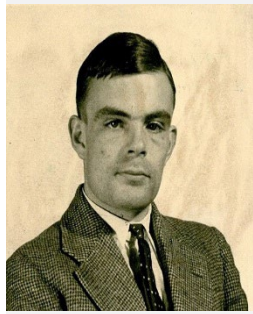


Noam Chomsky
(*1928)



John Backus
(1924-2007)
Turing Award 1977

## Berechenbarkeit (*computability*)



Alan Turing
(1912-1956)

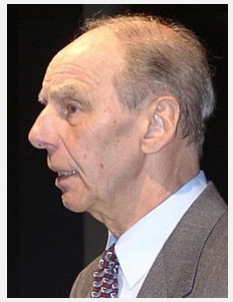


Alonzo Church
(1903-1995)



Kurt Gödel
(1906-1978)

## Komplexität



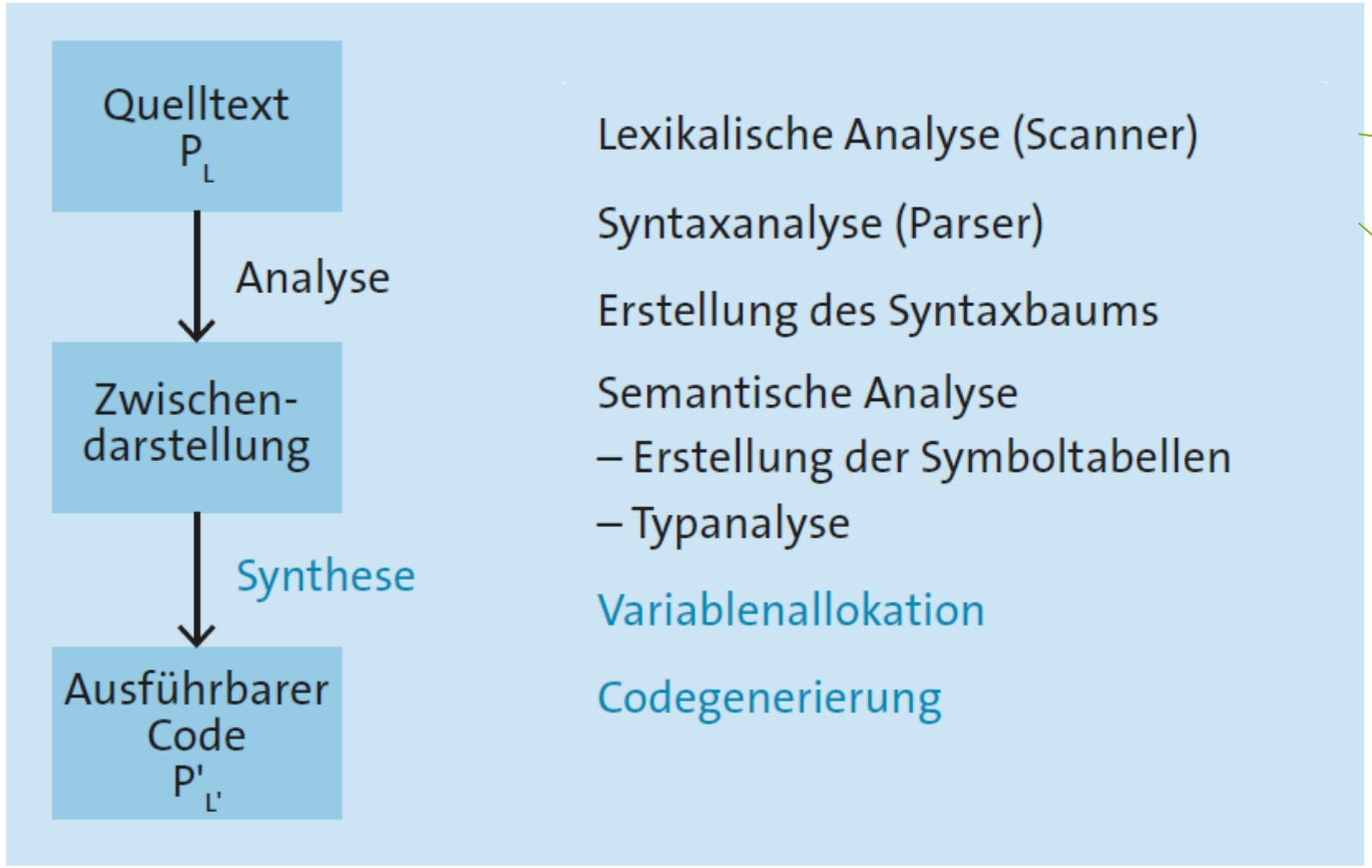
Juri Hartmanis
(1928-2022)



Richard Stearns
(*1936)

*„On the computational complexity of algorithms“, 1965*
*Turing Award 1993*

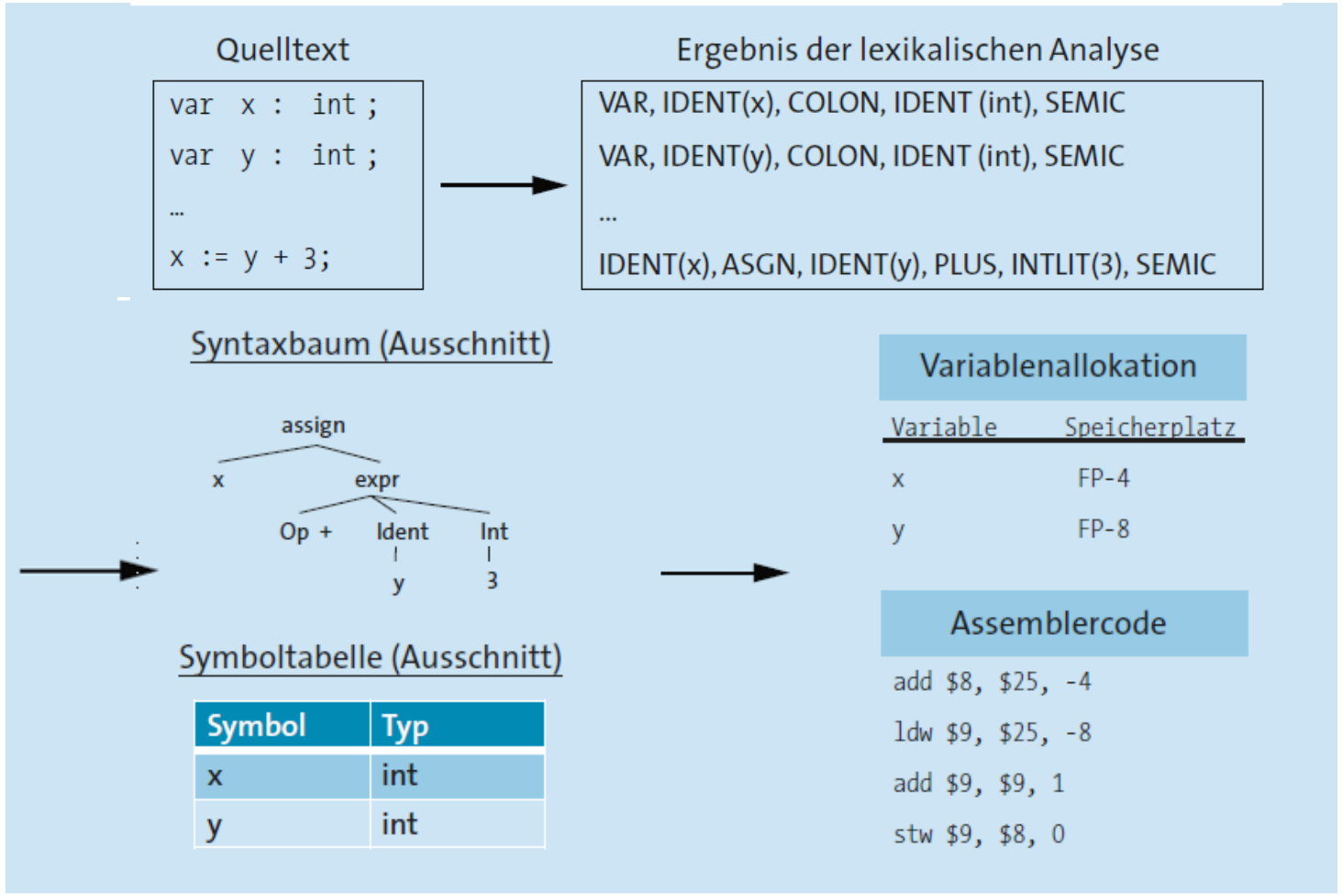
# Phasenmodell eines Compilers



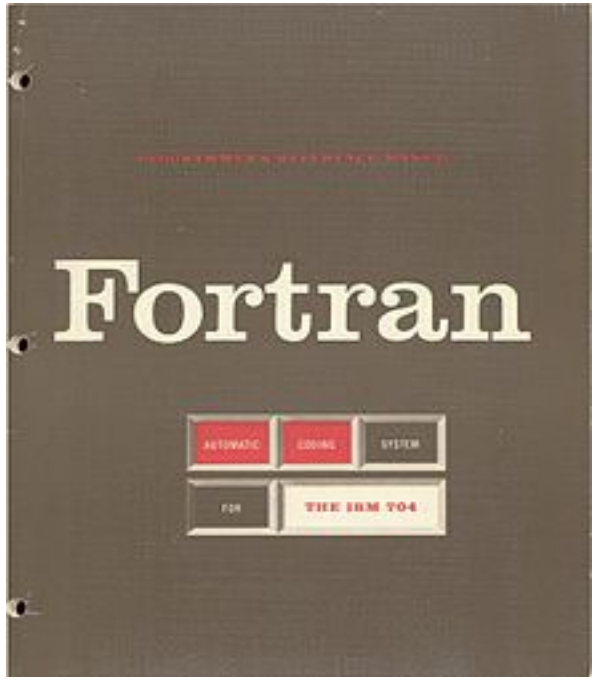

Kapitel 3
Reguläre Ausdrücke und Automaten

Kapitel 4: Kontext-freie Sprachen
Kapitel 5: Syntaxanalyse

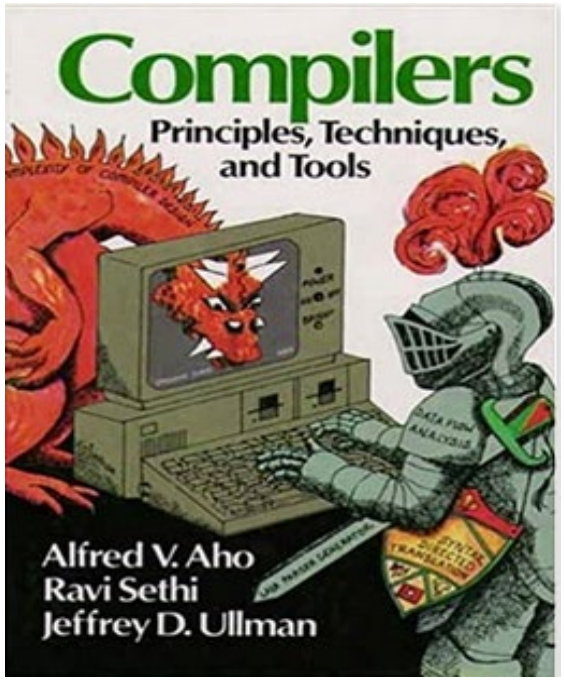
# Beispiel


Quelltext
var x : int ;
var y : int ;
…
x := y + 3;
Ergebnis der lexikalischen Analyse
VAR, IDENT(x), COLON, IDENT (int), SEMIC
VAR, IDENT(y), COLON, IDENT (int), SEMIC
...
IDENT(x), ASGN, IDENT(y), PLUS, INTLIT(3), SEMIC
Syntaxbaum (Ausschnitt)
assign
x
expr
Op +
Ident
y
Int
3
Symboltabelle (Ausschnitt)
Symbol
Typ
x
int
y
int
Variablenallokation
Variable
Speicherplatz
x
FP-4
y
FP-8
Assemblercode
add $8, $25, -4
ldw $9, $25, -8
add $9, $9, 1
stw $9, $8, 0

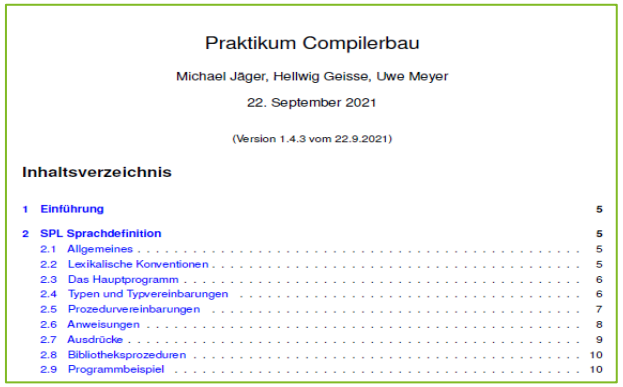
# Theorie und Praxis




1956: Keine Theorie → 54 Mannjahre Aufwand




1985: Der Compilerbauer kämpft mit den Waffen der theoretischen Informatik gegen die Komplexität des Compilers.



Praktikum Compilerbau

Michael Jäger, Hellwig Geisse, Uwe Meyer

22. September 2021

(Version 1.4.3 vom 22.9.2021)

**Inhaltsverzeichnis**

1 Einführung 5
2 SPL Sprachdefinition 5
2.1 Allgemeines 5
2.2 Lexikalische Konventionen 5
2.3 Das Hauptprogramm 6
2.4 Typen und Typvereinbarungen 6
2.5 Prozedurvereinbarungen 7
2.6 Anweisungen 8
2.7 Ausdrücke 9
2.8 Bibliotheksprozeduren 10
2.9 Programmbeispiel 10

Bison

JFlex

CUP LALR Parser Generator for Java™

2025: Studierende der THM entwickeln Tools, mit denen Sie Teile eines Compilers in 4 Monaten implementieren.